

Type inference for object definitions

Alex Gilding (Perforce UK)

Jens Gustedt (INRIA France)

2022-04-08

org:	ISO/IEC JCT1/SC22/WG14	document:	N2953
	... WG21 C and C++ liaison		P2305
target:	IS 9899:2023	version:	7
date:	2022-04-08	license:	CC BY (https://creativecommons.org/licenses/by/4.0/)

0.1. Abstract

We propose the inclusion of the so-called `auto` feature for variable definitions into C. This feature allows declarations to infer types from the expressions that are used as their initializers. This is part of a series of papers for the improvement of type-generic programming in C that has been introduced in N2890 and is continued with a series of papers that only concern object definitions N2952.

0.1. Summary of Changes

- N2953 (v7/R6)
 - remove type inference for function return types and function parameters
 - refocus type inference for objects to exactly match existing C practice
 - avoid type ambiguities by restricting to a single declarator per declaration
- N2923 (v6/R5)
 - move handling of `auto` parameters to N2924
 - make type declarations in underspecified declarations undefined behaviour
 - add Joseph Myers's wording for the `typeof` specifier, but make it its own sub-phrase
 - address an ambiguity concerning the scope of `auto` variables
 - make clear that shadowing by `auto` only concerns ordinary identifiers
 - minor text adaptations
- N2891 (v5/R4)
 - straighten the proposed text and move discussion of all special cases (bit-fields, compatible types, VM) into non-normative text such as notes and examples
- N2735 (v4/R3)
 - some word smithing
- N2697 (v3/R2)
 - remove a requirement on qualifiers and atomic derivation that was too restrictive
- N2674 (v2/R1)
 - changes the rules for type inference to use a `typeof` specifier. This simplifies the rules and brings the defined semantics in line with C++.
- N2632
 - original proposal

1. Introduction

Defining a variable in C requires the user to name a type. However when the definition includes an initializer, it makes sense to derive this type directly from the type of the expression used to initialize the variable. This feature has existed in C++ since C++11, and is implemented in GCC, Clang, and other GNU C compatible compilers using the `__auto_type` extension keyword. `__auto_type` is a much more limited feature than C++ `auto`, the latter of which is built on top of template type deduction rules. We propose to standardize the existing C extension practice directly. Any valid C construct using this syntax will also be valid and hold the same meaning within the broader semantics of the C++ feature.

This paper is based on N2952 which lays the ground work for the common syntax terminology that is needed for this paper here (N2953) and for a paper `constexpr` on object definitions (N2954).

2. Rationale

In N2890 it is argued that the features presented in this paper are useful in a more general context, namely for the combination with lambdas. We will not repeat this argumentation here, but try to motivate the introduction of the `auto` feature as a stand-alone addition to C. In accordance with C's syntax for declarations and in extension of its semantics, C++ has a feature that allows to infer the type of a variable from its initializer expression.

```
auto y = cos(x);
```

This eases the use of type-generic functions because now the return value and type can be captured in an auxiliary variable, without necessarily having the type of the argument, here `x`, at hand. That feature is not only interesting because of the obvious convenience for programmers who are perhaps too lazy to lookup the type of `x`. It can help to avoid code maintenance problems: if `x` is a function parameter for which potentially the type may be adjusted during the lifecycle of the program (say from `float` to `double`), all dependent auxiliary variables within the function are automatically updated to the new type.

This can even be used if the return type of a type-generic function is just an aggregation of several values for which the type itself is just an uninteresting artefact:

```
#define div(X, Y) \
    _Generic((X)+(Y), \
        int: div, \
        long: ldiv, \
        long long: lldiv) \
    ((X), (Y))

// int, long or long long?
auto res = div(38484848448, 448484844);
auto a = b * res.quot + res.rem;
```

An important restriction for the coding of type-generic macros in current C is the impossibility to declare local variables of a type that is dependent on the type(s) of the macro argument(s). Therefore, such macros often need arguments that provide the types for which the macro was evaluated. This not only inconvenient for the user of such macros but also an important source of errors. If the user chooses the wrong type, implicit conversions can impede on the correctness of the macro call.

For type-generic macros that declare local variables, `auto` can easily remove the need for the specification of the base types of the macro arguments:

```

#define dataCondStoreTG(P, E, D)      \
do {                                  \
    auto* _pr_p = (P);                \
    auto _pr_expected = (E);          \
    auto _pr_desired = (D);           \
    bool _pr_c;                       \
    do {                              \
        mtx_lock(&_pr_p->mtx);         \
        _pr_c = (_pr_p->data == _pr_expected); \
        if (_pr_c) _pr_p->data = _pr_desired; \
        mtx_unlock(&_pr_p->mtx);       \
    } while(!_pr_c);                 \
} while (false)

```

Cs declaration syntax currently already allows to omit the type in a variable definition, as long as the variable is initialized and a storage-class specifier (such as `auto` or `static`) disambiguates the construct from an assignment. In previous versions of C the interpretation of such a definition had been `int`; since C11 this is a constraint violation. We will propose to partially align C with C++, here, and to change this such that the type of the variable is inferred from the type of the initializer expression.

We achieve this by standardizing the existing practice in the GNU C dialect provided by the `__auto_type` specifier exactly. This is a strict subset of allowed C++11 behaviour. We expect and hope that implementers will treat incompatibilities with extended C++ declaration syntax (such as `auto const *`) as QoI bugs and implement these as extensions, establishing practice and experience for the C-side interpretation of such declarations. Standardizing the base value-only feature is a necessary basis to allow implementations to build beyond it with extended declarations.

3. Proposal

We propose standardizing the GNU C feature exactly, except for possibly changing the use of the extended specifier `__auto_type` to the existing specifier `auto`. The GNU C feature (since version 11) has clear semantics which can be expressed entirely in terms of the adopted `typeof` feature: the inferred type for a given initializer (`init`) is, exactly, `typeof((0, init))`. Namely the type is the type of the expression after lvalue, array-to-pointer or function-to-pointer conversion. Possible qualifiers in case `init` is an lvalue with a qualified type are dropped by that mechanism.

For example:

```

void foo (int x, int const y) {
    __auto_type a = x;
    __auto_type b = y;
    int * c = &a;
    int * d = &b; // OK
}
void bar (int x, int const y) {
    typeof(x) a = x;
    typeof(y) b = y;
    int * c = &a;
    int * d = &b; // not OK, qualifier discarded, GCC warns/errors
}

```

The feature is more limited than generic C declarations and than the corresponding C++ feature. A declaration using `__auto_type` must be initialized, must only consist of a single declarator, and may not have any part of its type specified at all; it must infer the entire object type from the initializer value, and cannot therefore be used in

combination with `*` or `[]` the way `auto` can in C++; there must be no type specifiers in the sequence apart from the `auto`.

The `auto` used as a complete type specifier may still be used in conjunction with qualifiers, attributes and with other storage-class specifiers:

```
void baz (int x, int const y) {
    __auto_type const a = x;
    __auto_type b = y;
    static __auto_type c = 1ul; // OK
    int * pa = &a;              // not OK
    int const * pb = &b;         // OK

    int * pc = &c;              // not OK, incompatible with unsigned long *
}
```

In order to avoid either specifying the wording for “same type”, which caused difficulty in accepting revision 5 of this proposal, or allowing different variables with the same syntactic specifier to infer different object types, we propose adding a new syntax constraint to exactly match the current GNU C behaviour that allows only one declarator per whole declaration using `__auto_type`. We expect implementations to gradually establish practice for how this rule should be relaxed as users explore the design space. As with the restriction against partially-specified types, we hope that implementations will support a more complete, C++-like extended feature that builds on current practice as users start to demand it, but do not standardize invention.

3.1. Alternative keywords

The original keyword that GNU C uses for this feature is `__auto_type` whereas C++ already has a mostly equivalent feature that uses the existing `auto` specifier. We leave the choice between the use of the two keywords open; in the proposed wording we use `AUTOTYPE` as a placeholder.

3.2. Direct coding instead of the `__auto_type` feature

C has accepted final wording for `typeof` and it is therefore now possible to declare an object in terms of the type of its initializer by writing:

```
typeof(((void)0, init)) var = init;
```

The main problem with this is the repetition. There is a maintenance burden to any use outside of macro expansion; for the case that `init` has a VM type, there is a potential side effect repetition; and readability is harmed for nontrivial expressions (which may be quite long). Practice shows that implementations do not need this repetition. They already know what the type of an initializing expression is, and are able to insert it into the specifiers implicitly. Therefore, the language should not force the use of a repeating construct when one is not necessary.

4. Impact

Implementation burden for this feature is low. Conforming C implementations are already able to delay fixing the type of a variable being declared until after seeing the initializer, as this is required for unspecified-array-size, where the element type of the array is known but the number of elements (and thus the complete type of the array object) is only known after the entire initializer right-hand-side has been seen. This feature therefore requires only a relatively minor change to existing machinery required for a conforming implementation. There is no ABI or runtime impact. The feature is purely syntactic.

4.1. Bit-fields

The definition of bit-fields in C is underspecified, in that their types are only known if an lvalue expression of a bit-field additionally undergoes integer promotion. If no such promotion is performed, for example in a `_Generic` or comma expression, implementations diverge in their interpretation of the standard. Some always produce one of the types `bool`, `signed int` or `unsigned int`, others produce some implementation defined types that reflect the width of the bit-field. The latter are not integer types in the sense of the standard because they only have to convert under promotion and need not to have any other property of integers, and, usually don't have documented declaration syntax. It is not the place of this proposal here to sort out this inconsistency between different interpretations of the standard. This proposal specifies the feature in terms of the type produced by lvalue conversion, array-to-pointer and function-to-pointer conversion; whatever an implementation does there for bit-fields, should be good enough.

4.2. Combined definitions and compatible types

The semantics of underspecified declarations become complicated if they contain definitions for several objects where the inferred type has to be consistent. In revision 5 the choice was that inferred types have to be the same, only having compatible types is not sufficient. This is particularly important for integer types, where mixing different enumeration types would have an ambiguity which type is chosen.

This partially caused revision 5 to be rejected at the January 2022 WG14 meeting and therefore the proposal resolves this difficulty by eliminating the syntax that would allow for any ambiguity here. Declarations using inferred types must now form separate declarations in line with existing GNU C practice.

4.3. Combined definitions and variably modified types

Revision 5 included a consistency problem in that different types within the same definition could not be checked for consistent VM elements. This inconsistency is removed by not supporting multiple declarations within a single statement, so there is no longer any constraint to satisfy.

This behavior is ensured by the wording for underspecified declarations as it is proposed in N2952.

4.4. Specifiers

`AUTOTYPE` can be used in combination with other storage-class specifiers such as `static`, `register`, etc., the only one with which is not allowed to combine is `typeof`.

`auto` now has no effect if it is not used to infer a type. We expect implementations to continue to warn as a matter of QoI. This does not break any existing conforming code.

4.5. Scope

Existing practice in GNU C is that the identifier being declared has scope beginning after the end of the full-declaration, as opposed to all other identifiers which enter scope at the end of their declarator. This behavior is ensured by the wording for underspecified declarations as it is proposed in N2952.

4.6. Ambiguities with type definitions

Since identifiers may be redeclared in inner scopes, ambiguities with identifiers that are type definitions could occur. We resolve that ambiguity by reviving a rule that solved the same problem when C still had the implicit `int` rule. This is done in 6.7.8 p3 (Type definitions) by adding the following phrase:

If the identifier is redeclared in an inner scope the inner declaration shall not be underspecified.

5. Implementation Experience

`__auto_type` is implemented by most (all?) compilers implementing the GNU C dialect or aiming for GCC compatibility. A non-exhaustive list includes: GCC, Clang, Intel CC, Helix QAC, Klocwork, armCC. It is generally used to implement type-generic macros in library headers. It does not appear to be widely used by developers in application code but is heavily tested by virtue of its appearance in Standard header implementations.

Many compilers exist which borrow components from GCC or Clang, and therefore inherit this feature intentionally or unintentionally.

A more comprehensive feature exists in C++ since C++11, which is based on template type deduction rules and can therefore use `auto` to infer parts of a partially-specified type, such as specifying that a declaration creates a pointer or reference but not what it is a pointer or reference to. This is in near-universal use by millions of C++ developers every day.

This `auto` feature from C++ is also implemented by clang for their C frontend. In addition, clang also extends the `__auto_type` feature such that it covers the same semantics as their `auto`, thus presenting essentially a single extension that can be spelled with two different keywords, `auto` and `__auto_type`.

Specifying a feature closer to the C++ specifier would require substantial original wording in the Standard since C does not include templates, which the C++ feature is defined in terms of. Usability experience from C++ might set user expectation to be able to write `auto * foo = ...`. Therefore the text leaves room for extensions; declarations with several declarators or with pointer derivations, for example, are undefined and not constraints. There is at least already one implementation that provides such a wider functionality, clang, and our intent is not to constrain these too much.

6. Proposed wording

Changes are proposed against the wording in C23 draft n2731 to which the accepted changes concerning keywords and N2952 have been added. Green and underlined text is new text. The token `AUTOTYPE` has to be replaced by either `__auto_type` or `auto`, whichever is chosen by WG14 to represent the feature.

6.1. Linkage of identifiers (6.2.2)

Modify

5 If the declaration of an identifier for a function has no storage-class specifier, its linkage is determined exactly as if it were declared with the storage-class specifier `extern`. If the declaration of an identifier for an object has file scope and no storage-class specifier or only the specifier `AUTOTYPE`, its linkage is external.

6.2. Keywords (6.4.1)

If necessary, add `__auto_type` to the list of keywords. If the choice falls on using `auto` instead, no change is necessary.

6.3. Storage-class specifiers (6.7.1)

If necessary, add `__auto_type` to the list of storage-class specifiers. If the choice falls on using `auto` instead, no change is necessary.

Modify the constraints section

Constraints

2 At most, one storage-class specifier may be given in the declaration specifiers in a declaration, except that `thread_local` may appear with `static` or `extern`, and that `AUTOTYPE` may appear with all others but with `typedef`.¹²⁷⁾

3 In the declaration of an object with block scope, if the declaration specifiers include `thread_local`, they shall also include either `static` or `extern`. If `thread_local` appears in any declaration of an object, it shall be present in every declaration of that object.

4 `thread_local` shall not appear in the declaration specifiers of a function declaration. `AUTOTYPE` shall only appear in the declaration specifiers of an identifier with file scope if the type is to be inferred from an initializer.

Add a new paragraph

9 If `AUTOTYPE` appears with another storage-class specifier, or if it appears in a declaration at file scope it is ignored for the purpose of determining a storage duration or linkage. It then only indicates that the declared type may be inferred.

Modify the forward references section

Forward references: type definitions (6.7.8), type inference (6.7.10).

6.4. Type specifiers (6.7.2)

Modify the beginning of the following paragraph of the Constraints section

~~At~~ Except where the type is inferred (6.7.10), at least one type specifier shall be given in the declaration specifiers in each declaration, ...

Add a new paragraph in the Semantics section after paragraph 4

4' For a declaration such that the declaration specifiers contain no type specifier a mechanism to infer the type from an initializer is discussed in 6.7.10. In such a declaration, optional elements, if any, of a sequence of declaration specifiers appertain to the inferred type (for qualifiers and attribute specifiers) or to the declared objects (for alignment specifiers).

6.5. Type definitions (6.7.8)

Add to the end of paragraph 3 of the Semantics section

... A typedef name shares the same name space as other identifiers declared in ordinary declarators. If the identifier is redeclared in an enclosed block the inner declaration shall not be such that the type is inferred.

6.6. Declarations (6.7)

Add a new normative clause

6.7.10 Type inference

Constraints

1 A declaration for which the type is inferred shall contain the storage-class specifier `AUTOTYPE`.

Description

2 For such a declaration that is the definition of an object the init-declarator shall have one of the forms

```
direct-declarator = assignment-expression  
direct-declarator = { assignment-expression }  
direct-declarator = { assignment-expression , ... }
```

The declared type is the type of the assignment expression after lvalue, array to pointer or function to pointer conversion, additionally qualified by qualifiers and amended by attributes as they appear in the declaration specifiers, if any.^{FNT1)} If the direct declarator is not of the form

```
identifier attribute-specifier-sequenceopt
```

possibly enclosed in balanced pairs of parenthesis the behavior is undefined.

^{FNT1)} The scope rules as described in 6.2.1 also prohibit the use of the identifier of the declarator within the assignment expression.

6.7. Non-normative additions

6.7.1. note and examples

Additionally, add the following non-normative text to the new clause.

3 **NOTE** Such a declaration that also defines a structure or union type violates a constraint. Here, the identifier `a` which is not ordinary but in the name space of the structure type is declared.

```
AUTOTYPE p = (struct { int a; } *)0;
```

Even a forward declaration of a structure tag

```
struct s;  
AUTOTYPE p = (struct s { int a; } *)0;
```

would not change that situation. A direct use of the structure definition as the type specifier ensures the validity of the declaration.

```
struct s { int a; } * p = 0;
```

4 **EXAMPLE 1** Consider the following file scope definitions:

```
static AUTOTYPE a = 3.5;  
AUTOTYPE p = &a;
```


They are interpreted as if they had been written as:

```
static double a = 3.5;
double * p = &a;
```

So effectively `a` is a `double` and `p` is a `double*`. Note that the restrictions on the syntax of such declarations does not allow the declarator to be `*p`, but that the final type here nevertheless is a pointer type.

5 **EXAMPLE 2** The scope of the identifier for which the type is inferred only starts after the end of the initializer (6.2.1), so the assignment expression cannot use the identifier to refer to the object or function that is declared, for example to take its address. Any use of the identifier in the initializer is invalid, even if an entity with the same name exists in an outer scope.

```
{
    double a = 7;
    double b = 9;
    {
        double b = b * b;           // undefined, uses uninitialized variable
        without address
        printf("%g\n", a);           // valid, uses "a" from outer scope,
        prints 7
        AUTOTYPE a = a * a;         // invalid, "a" from outer scope is
        already shadowed
    }
    {
        AUTOTYPE b = a * a;         // valid, uses "a" from outer scope
        AUTOTYPE a = b;             // valid, shadows "a" from outer scope
        ...
        printf("%g\n", a);         // valid, uses "a" from inner scope,
        prints 49
    }
    ...
}
```

6 **EXAMPLE 3** In the following, declarations of `pA` and `qA` are valid. The type of `A` after array-to-pointer conversion is a pointer type, and `qA` is a pointer to array.

```
double A[3] = { 0 };
AUTOTYPE pA = A;
AUTOTYPE qA = &A;
```

7 **EXAMPLE 4** Type inference can be used to capture the type of a call to a type-generic function. It ensures that the same type as the argument `x` is used.

```
#include <tgmath.h>
AUTOTYPE y = cos(x);
```

If instead the type of `y` is explicitly specified to a different type than `x`, a diagnosis of the mismatch is not enforced.

8 EXAMPLE 5 A type-generic macro that generalizes the `div` functions (7.22.6.2) is defined and used as follows.

```
#define div(X, Y) _Generic((X)+(Y), int: div, long: ldiv, long long:
    lldiv)((X), (Y))
AUTOTYPE z = div(x, y);
AUTOTYPE q = z.quot;
AUTOTYPE r = z.rem;
```

9 EXAMPLE 6 Definitions of objects with inferred type are valid in all contexts that allow the initializer syntax as described. In particular they can be used to ensure type safety of `for`-loop controlling expressions.

```
for (AUTOTYPE i = j; i < 2*j; ++i) {
    ...
}
```

Here, regardless of the integer rank or signedness of the type of `j`, `i` will have the non-atomic unqualified type of `j`. So, after lvalue conversion and possible promotion, the two operands of the `<` operator in the controlling expression are guaranteed to have the same type, and, in particular, the same signedness.

6.7.2. Storage-class specifiers (6.7.1)

Adapt the changed p6 as of N2952

6 Storage-class specifiers specify various properties of identifiers and declared features; storage duration (`static` in block scope, `thread_local`, `auto`, `register`), linkage (`extern`, `static` in file scope, `typedef`) and type (`typedef`, `AUTOTYPE`). The meanings of the various linkages and storage durations were discussed in 6.2.2 and 6.2.4, `typedef` is discussed in 6.7.8, type inference using `AUTOTYPE` is specified in 6.7.10.

6.7.3. Undefined behavior (J.2)

Add two new items to the list

- ...
- The initializer for an aggregate or union, other than an array initialized by a string literal, is not a brace-enclosed list of initializers for its elements or members (6.7.9).
- A declaration for which a type is inferred contains pointer, array or function declarators (6.7.10).
- A declaration for which a type is inferred contains no or more than one declarators (6.7.10).
- An identifier with external linkage is used, but in the program there does not exist exactly one external definition for the identifier, or the identifier is not used and there exist multiple external definitions for the identifier (6.9).
- ...

6.7.4. Common extensions (J.5)

Add a new clause

J.5.18 Type inference

1 A declaration for which a type is inferred (6.7.10) may additionally accept pointer declarators, function declarators and may have more than one declarator.

7. Questions to WG14

7.1. Keyword

Does WG14 prefer the use keyword `auto` for type inference as proposed in N2953?

7.2. Acceptance

Does WG14 want to include the type inference feature of N2953 together with the underspecified declaration feature of N2952 into C23?

8. References

- C23 (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf>)
- Improve type generic programming (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2890.pdf>)
- Not so magic typeof (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2927.htm>)
- C++11 (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3337.pdf>)
- GCC typeof and `__auto_type` (<https://gcc.gnu.org/onlinedocs/gcc/Typeof.html#Typeof>)